



Time your commands using the time command

First run

```
> time ffmpeg ...
```

```
real 0m 2.06s
```

← Total observed time

```
user 0m 0.17s
```

← Time in userspace (running the program and shared libs)

```
sys 0m 0.26s
```

← Time in kernel space (accessing files, accessing device data)

Second run (program and libraries already in file cache)

```
> time ffmpeg...
```

```
real 0m 0.66s
```

← Less waiting time!

```
user 0m 0.17s
```

```
sys 0m 0.25s
```

real = user + sys + waiting time (at least on single core machines)

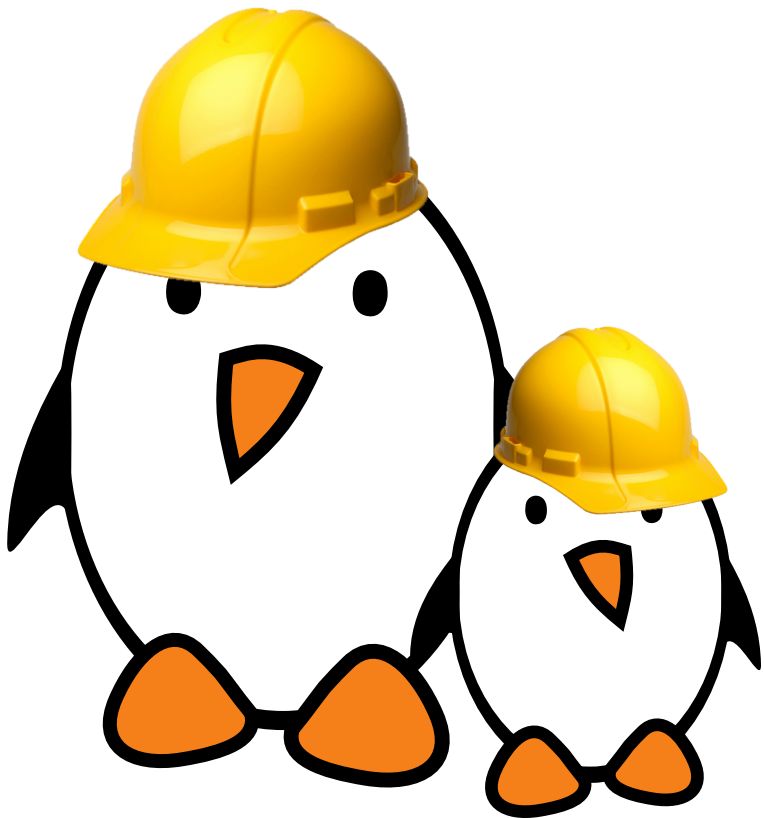


Your program cannot run faster than user + sys (unless you optimize the code)

This gives you the best time that can possibly be achieved (with the fastest storage).



Practical lab - Toolchain optimizations



- ▶ Measure filesystem and `ffmpeg` binary size.
Time the execution of the application.
- ▶ Re-compile the root filesystem using a Thumb2 toolchain
- ▶ Re-compile the root filesystem with the *Musl* C library instead of *uClibc*
- ▶ Find the best toolchain in terms of size and execution time.
- ▶ Have Buildroot generate an external toolchain (*SDK*)



Lessons from labs: ARM vs Thumb2 (32 bit only)

- ▶ Testcase: root filesystem with `ffmpeg` and associated libraries (from our training labs), with `uClibc`
- ▶ Compiled with `gcc 10.3`, generating *ARM* code: Total filesystem size: 17.9 MB
`ffmpeg` size: 239 KB
- ▶ Compiled with `gcc 10.3`, generating *Thumb2* code: Total filesystem size: 14.5 MB
(-19 %) `ffmpeg` size: 191 KB (-20 %)
- ▶ Performance aspect: performance apparently slightly improved with Thumb2 (about 2 %, but there are slight variations in measured execution time, for one run to another).



Lessons from labs: musl vs uClibc

Replacing *uClibc* by *musl* in our video player lab, keeping *Thumb2*. Here are data from an earlier run of our labs:

- ▶ Total system size with *uClibc*: 14.3 MB
- ▶ Total system size with *Musl*: 14.4 MB
- ▶ uClibc saves 80 KB (useful), but otherwise no other significant change in filesystem and code size. Not a surprise when the system is mostly filled with binaries relying on shared libraries.

Switching to *Musl* as it is supposed to allow for smaller static binaries, which will be useful in later labs.